Bài thực hành số 7

A. Mục tiêu

+ Truyền dữ liệu giữa các component
+ Tổ chức và sử dụng Service trong Angular
+ Hiểu và sử dụng được HttpClient để truy xuất dữ liệu từ API bên ngoài ứng dụng.

B. Nội dung thực hành

- Giả lập API
- Xây dựng một lớp httpservice đọc dữ liệu từ API
- Kết nối dữ liệu vào ứng dụng StockMarket để thực hiện các chức năng CRUD với API

1. Hướng dẫn giả lập API:

Giả lập API sử dụng JSON Server

Cài đặt JSON Server: JSON Server được đóng gói như một NPM package. Vì vậy việc cài đặt có thể được thực hiện thông qua việc sử dụng gói Node.js manager:

```
npm install -g json-server
```

Tuỳ chọn -g sẽ giúp cho package được cài đặt ở level hệ thống.

Tạo JSON File: với tên data.json. File này chứa dữ liệu truy xuất bởi RESTFull API. Dữ liệu demo của file:

```
{
  "stocks": [
    {
      "id": 1,
      "name": "Test Stock Company",
      "code": "TSC",
      "price": "85",
      "previousPrice": "80",
      "exchange": "NASDAQ",
      "favorite": "false"
    },
    {
      "id": 2,
      "name": "Second Stock Company",
      "code": "SSC",
      "price": "10",
```

```
      "previousPrice": "20",
      "exchange": "NSE",
      "favorite": "false"
    },
    {
      "id": 3,
      "name": "Last Stock Company",
      "code": "LSC",
      "price": "876",
      "previousPrice": "765",
      "exchange": "NYSE",
      "favorite": "false"
    },
    {
      "name": "Last Stock Company",
      "code": "LSC",
      "price": "876",
      "previousPrice": "765",
      "exchange": "NYSE",
      "favorite": "false",
      "id": 4
    }
  ],
  "profile": {
    "name": "typicode"
  }
}
```

Khởi động Server: khởi động JSON server bằng việc chạy câu lệnh sau:

`json-server --watch data.json`

File data.json được truyền vào như một tham số trong câu lệnh trên, và option --watch được thêm vào nhằm đảm bảo server được start ở chế độ "watch", trong chế độ này, server sẽ "watch" cho những thay đổi của file và cập nhật kết quả vào API một cách phù hợp.

Test API bằng cách mở trình duyệt với url http://localhost:3000/ ta sẽ nhận được kết quả của file json đã tạo.

2. Xây dựng httpService như sau:

src > app > services > TS http-service.service.ts > ...

```ts
import { HttpClient, HttpHeaders } from '@angular/common/http';
import { Injectable } from '@angular/core';
import { Observable } from 'rxjs';

@Injectable({
  providedIn: 'root'
})
export class HttpServiceService {

  private REST_API_SERVER='http://localhost:3000';
  private httpOptions={
    headers: new HttpHeaders({
      'Content-Type': 'application/json',
    }),
  };

  constructor(private httpClient: HttpClient) { }
  public getStocks(): Observable<any>{
    const url=`${this.REST_API_SERVER}/stocks`;
    return this.httpClient.get<any>(url, this.httpOptions);
  }
  public postStock(body: any): Observable<any>{
    const url=`${this.REST_API_SERVER}/stocks`;
    console.log('postStock=', url);
    console.log('postStock: body', body);
    return this.httpClient.post<any>(url, body,this.httpOptions);
  }
}

```

3. Get component

src > app > stock > get-data > TS get-data.component.ts > ...

```ts
import { Component, OnInit } from '@angular/core';
import { HttpServiceService } from 'src/app/services/http-service.service';

@Component({
  selector: 'app-get-data',
  templateUrl: './get-data.component.html',
  styleUrls: ['./get-data.component.css']
})
export class GetDataComponent implements OnInit {

  constructor(private httpServerService: HttpServiceService) { }

  ngOnInit(): void {
    this.httpServerService.getStocks().subscribe((data)=>{
      console.log('stocks', data);
    })
  }
}
```

4. Post component

src > app > stock > post-data > TS post-data.component.ts > ...

```ts
import { Component, OnInit } from '@angular/core';
import { HttpServiceService } from 'src/app/services/http-service.service';

@Component({
  selector: 'app-post-data',
  templateUrl: './post-data.component.html',
  styleUrls: ['./post-data.component.css']
})
export class PostDataComponent implements OnInit {

  constructor(private httpServerService: HttpServiceService) { }

  ngOnInit(): void {
    const body={"name": "Last Stock Company",
    "code": "LSC",
    "price": "876",
    "previousPrice": "765",
    "exchange": "NYSE",
    "favorite": "false"};
    this.httpServerService.postStock(body).subscribe((data)=>{
      console.log('postStock', data);
    })
  }
}
```

5. Yêu cầu thêm:

- Viết thêm các phương thức update, delete, get… trong httpService để cập nhật và xóa một cổ phiếu
- Sử dụng httpService cho các thao tác hiển thị, thêm xóa sửa tìm kiếm cổ phiếu và mọi thay đổi sẽ cập nhật lên dữ liệu giả lập ở data.json